Technical Design

NestUp

Architecture, data model, business logic, and how it is built

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

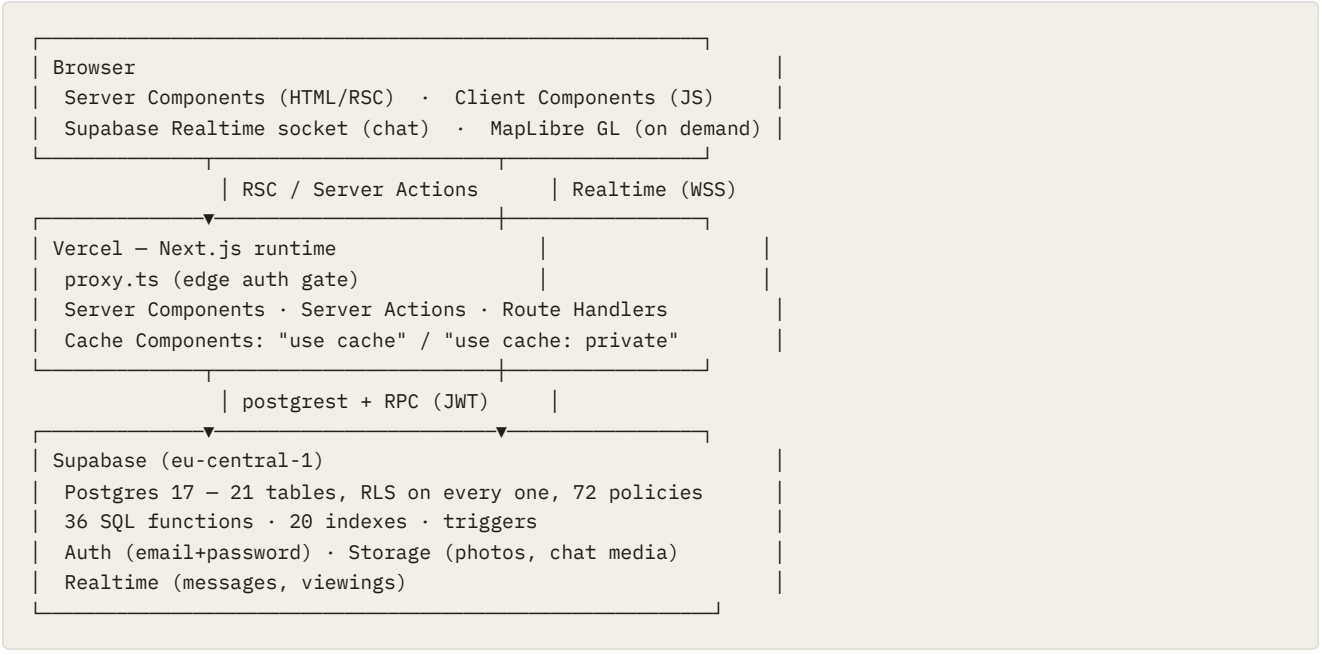
Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

Stack: Next.js 16.3.2 (App Router) · TypeScript · Supabase (Postgres + Auth + Realtime + Storage) · Tailwind CSS v4 · Vercel

---

1. Architecture overview



The central architectural decision is the following: the browser never queries the database as a trusted client. Every read passes through a Server Component and every write passes through a Server Action, and *underneath both of them*, Postgres Row Level Security decides what the caller is actually permitted to see or to modify. In other words, authorization is enforced in the database and not in the application. Therefore a bug in a page or in an action cannot expose the data of another member.

Why this stack was chosen

CHOICE	REASON
Next.js App Router	Server Components allow the data-heavy pages, such as a list of 20 rooms or a ranked deck, to render on the server and to send HTML instead of JSON followed by a client-side fetch. In addition, Server Actions remove the need to write an API layer manually for every mutation.
TypeScript	The domain contains many shapes which are nearly identical, such as `Profile`, `Listing` and `ConversationSummary`. Types catch exactly the confusions which a compiler is able to see. It is also required by the assignment.
Supabase	The decisive reason is Postgres with RLS. It provides a genuine relational database *together with* an authorization model which lives next to the data. Moreover, the fact that Auth, Storage and Realtime belong to the same product removes three separate integrations.
Zod	One schema is defined per input shape, and it is used both for validation and for deriving the TypeScript type. Consequently the two cannot drift apart.
Tailwind v4	Semantic design tokens are used (`bg-paper`, `text-ink`, `border-hairline`, `accent`) rather than raw palette colors, and this is what makes two themes possible without maintaining a second stylesheet.
MapLibre + CARTO	This combination is keyless and served through our own worker. Therefore there is no map vendor API key which could leak, and no dependency on a third-party CDN at runtime.

2. Folder structure

```
Final-Project/
├── app/                          # Next.js App Router
│   ├── (public)/                 # No session required
│   │   ├── page.tsx              #   / → redirects to /browse
│   │   └── browse/               #   Listings index, room page, "message the household"
│   ├── (app)/                    # Signed-in area (gated at the edge by proxy.ts)
│   │   ├── swipe/                #   Personalized deck
│   │   ├── chat/                 #   Inbox layout + thread
│   │   ├── profile/              #   Profile + edit
│   │   ├── people/[id]/          #   Another member's profile
│   │   ├── listing/              #   Create / edit a room
│   │   ├── settings/             #   Account, privacy, notifications, danger zone
│   │   └── loading.tsx           #   Route-group skeleton
│   ├── (auth)/                   # Login, signup, verify, forgot/reset password
│   ├── actions/                  # 14 Server Action modules — every mutation
│   ├── api/                      # Route Handlers (JSON, or non-action side effects)
│   ├── auth/                     # confirm (email links) · signout (303 → full reload)
│   └── layout.tsx                # Root layout: fonts, theme, nav, view transitions
├── components/                   # 99 components, grouped by feature
│   ├── auth/ (8)   chat/ (10)   listings/ (15)   map/ (11)
│   └── profile/ (24)  settings/ (8)  swipe/ (4)  ui/ (19)
├── lib/                          # 53 modules — domain logic, no JSX
│   ├── supabase/                 #   4 clients: server, browser, public (anon), middleware
│   ├── validation/               #   6 Zod schemas
│   ├── compatibility.ts          #   The scoring function
│   ├── swipe.ts / swipe-deck.ts#   Deck construction (client-safe / server-only)
│   ├── cache-tags.ts             #   Every cache tag in one place
│   └── auth.ts                   #   Session helpers + cached readers
├── supabase/
│   ├── migrations/               # 43 SQL migrations, all applied to production
│   └── templates/                # Auth email templates (versioned, not clicked into a UI)
├── tests/unit/                   # 97 files, 740 tests
├── scripts/                      # Seeding, geocoding, real-browser checks
└── docs/submission/              # These documents
```

The organizing rule is the following: `app/` decides *what a given URL renders*, `components/` is responsible for presentation, and `lib/` contains the domain logic which could be tested without a browser at all. Every module which touches `cookies()` is marked as `server-only`, so that it cannot be pulled into a client bundle by mistake.

Route groups, and the reason there are three of them

The groups `(public)`, `(app)` and `(auth)` do not appear in the URLs. They exist so that each set of pages can have its own layout and its own assumption regarding access. The `(app)` group is gated at the edge; the `(public)` group renders for anybody and reveals session-specific elements only inside a `<Suspense>` boundary; and the `(auth)` group has no navigation chrome at all.

---

3. Database design

The database contains 21 tables, with **RLS enabled on every single one of them**, 72 policies, 36 SQL functions and 20 indexes.

Core entities

TABLE	ROWS (PROD)	PURPOSE
`profiles`	845	One row per member, holding identity, how the member lives, and what the member expects from roommates. The primary key is `auth.users.id`.
`profile_details`	8	Private additional information: phone, contact email, social links, introduction template and visibility flags. It is a separate table because its RLS is stricter than that of `profiles`.
`listings`	824	A room in a shared apartment. It is owned by one member and co-owned through `listing_residents`.
`listing_residents`	1,290	The members who live in the household of a listing. This table is what turns a listing from the advertisement of one person into the advertisement of a household.
`listing_invites`	0	Co-poster invitations. **There is no insert, update or delete policy** on this table; it is written only by two SECURITY DEFINER functions.
`conversations`	29	One thread per (listing, seeker) pair, with a unique constraint on that pair.
`messages`	57	Chat messages, including a `client_id` which allows an idempotent retry.
`conversation_reads`	29	The read cursor of each member, which is what produces the unread counters.
`conversation_deletes`	12	The "delete chat" cutoff of each member. The semantics are those of WhatsApp, meaning that the history is hidden for one side only.
`viewings`	17	Apartment viewings which were proposed and approved inside a chat.
`swipes`	117	Like and skip decisions, unique on (seeker, listing), so that a room never returns to the deck.
`saved_listings`	33	Hearts, presented in the interface as "Liked".
`listing_views`	883	History of which listings a member opened.
`listing_dwell`	24	How long a card was actually looked at, which serves as an engagement signal.
`blocks`	2	A mutual block, which removes both directions from the decks and from the search.
`reports`	0	Reported members and listings.
`suspensions`	0	**There is no write policy at all** on this table; it is written only by `apply_report_suspension()`. Therefore a member cannot remove his or her own suspension.
`google_tokens`	0	OAuth tokens used for calendar export. Accessible to the owner only.
`app_config`	1	Server-side configuration values. It has no RLS policies, and it is reachable only through SECURITY DEFINER functions.
`auth_mail_throttle`	10	Rate limiting for outgoing authentication email.
`matches`	0	Reserved for mutual-match semantics; in practice it was superseded by conversations.

Design decisions which are worth defending

The column `profiles.user_id` is simultaneously the primary key and a foreign key to `auth.users`. There is no separate profile identifier. As a result, every RLS policy in the schema is a comparison against `auth.uid()` without any join, and account deletion cascades correctly by construction.

Two tables are used for a single person, namely `profiles` and `profile_details`. The table `profiles` is readable by other members, since it is what the deck and the room page render. The table `profile_details` holds the phone number and the contact email and is accessible to the owner only, together with a `public_profile_details()` function which returns only those fields that the owner explicitly chose to expose. The consequence of this split is that the *default* is privacy: a new column added to `profile_details` does not accidentally become readable by the entire world.

The columns `household_size` and `household_gender` on `listings` are denormalized. They are maintained by triggers over `listing_residents`. They exist because both the browse filter and the deck require them on every row, and computing them per row would transform a single query into 800 queries.

Deletion is soft rather than hard. This is implemented through `listings.removed_at`, `listings.taken_at` and `conversation_deletes.cleared_at`. A room which no longer exists must not take its conversations with it, because the people who discussed that room still own that history. This is also the reason that there are two SELECT policies on `listings`: one for the public list, and one for the case of "I am linked to this listing through a conversation".

The column `messages.client_id`. The browser generates a UUID before sending the message. A retry which carries the same identifier collides with a unique index and returns the existing row instead of posting the message twice. This is precisely what makes the optimistic user interface safe on an unstable connection.

Indexes

There are 20 indexes, placed at the points where the query planner would otherwise perform a scan: `swipes(seeker_id, listing_id)`, `messages(conversation_id, created_at)`, `listings(is_active, city)`, `listing_views(user_id, viewed_at)`, `saved_listings(user_id)`, `blocks(blocker_id)`, `viewings(conversation_id)` and others. They are discussed in greater detail in the Scale document.

---

4. Key CRUD operations

Every mutation is implemented as a **Server Action**. There are no client-side database writes anywhere in the product.

ENTITY	CREATE	READ	UPDATE	DELETE
Profile	`saveProfileAction` (first save = onboarding)	`getCachedOwnProfile`, `/people/[id]`	`saveProfileAction`, `saveAboutAction`	`deleteAccountAction` (RPC `delete_own_account`)
Listing	`saveListingAction`	`queryListings`, `getListing`	`saveListingAction`	`removeListingAction` (soft), `setListingActiveAction` (pause), `markTakenAction`
Photos	`checkAndUploadPhotoAction` (AI-checked, then Storage)	Public bucket URL	Reorder in form	Remove in form
Swipe	`recordSwipeAction`	`getCachedDeck`	Upsert on conflict	— (a decision is permanent)
Saved	`setSavedAction`	`getSavedListingIds`	—	`setSavedAction(false)`
Conversation	`findOrCreateConversation`	`getCachedInbox`, thread page	`markReadAction`	`deleteConversationAction` (per-member cutoff)
Message	`sendMessageAction`	Thread page + Realtime	—	—
Viewing	`proposeViewingAction`	Thread page	`respondViewingAction`	Cancel via status
Invite	`inviteRoommates` (RPC)	`getPendingInvites`	`respondToInviteAction` (RPC)	—
Moderation	`reportAction`, `blockUserAction`	`getBlocked`	—	`unblockUserAction`

The reason for preferring Server Actions over REST for mutations is that an action is a typed function which the component imports directly. Hence there is no URL which must be kept synchronized, no request or response shape which must be written by hand, and no possibility of calling the action without the framework attaching the session to it. The four Route Handlers which nevertheless do exist are exactly those cases in which an action does not fit.

5. API surface

Route Handlers (`app/api/` , `app/auth/`)

ROUTE	METHOD	WHY IT IS A HANDLER AND NOT AN ACTION
`/api/listings`	GET	It is queried by client-side code and therefore requires a JSON response
`/api/listings/pins`	GET	All placed rooms for the map, approximately 150 KB, fetched only when the map is opened
`/api/listings/[id]/invites`	GET	Co-poster state for the listing form
`/api/invites` , `/api/invites/[id]`	GET/POST	The invitation list and the response to an invitation
`/api/places`	GET	Proxies Overpass for cafés and shops near a room, and therefore **keeps the upstream call on the server side**
`/api/google/connect` , `/api/google/callback`	GET	The OAuth redirect flow, which requires the browser to be redirected to this location
`/auth/confirm`	GET	The destination at which every emailed authentication link arrives
`/auth/signout`	POST	It must answer with **303**, so that the browser performs a full document load and consequently discards all client caches

API routes verify authorization inside the handler itself. The edge proxy protects page routes only, because an API caller must receive a JSON 401 response and not an HTML redirect to `/login` .

SQL functions (RPC)

There are 36 functions. The important ones are declared as `SECURITY DEFINER` , meaning that they run with elevated rights precisely in order that the *table* itself may have no write policy whatsoever:

- `my_conversations()` and `my_unread_count()` , which produce the inbox and the badge and are scoped by RLS internally
- `mark_conversation_read()` and `clear_conversation()` , which handle the read cursor and the per-member delete
- `delete_own_account(p_heir)` , which deletes the account and **transfers a shared listing to a roommate within the same transaction**
- `invite_listing_roommates()` and `respond_to_listing_invite()` , which are the only writers to `listing_invites`
- `apply_report_suspension()` , which is the only writer to `suspensions`
- `public_profile_details()` , which returns only those fields that a member chose to expose
- `blocked_user_ids()` , `is_blocked()` , `is_suspended()` and `linked_to_listing()` , which serve as policy helpers

This is the pattern which is most worth explaining: **when a table must only ever be written in one specific manner, it should be given no write policy at all and exactly one definer function.** Under this arrangement there is simply no path which leads to an incorrect write, which is a stronger guarantee than a policy that attempts to enumerate every possible incorrect write.

Third-party APIs

Everything the application calls which it does not itself host. **None of them is paid**, and only three require a credential at all, which is the reason the whole project can be deployed without an account anywhere beyond Supabase and a Google API key.

API	WHAT IT IS USED FOR	CREDENTIAL	WHERE
Supabase — Postgres, Auth, Realtime, Storage	The backbone: every query, the session, the chat socket, and the `avatars`, `listing-photos` and `chat-images` buckets	Publishable key in the browser; service-role key on the server only	Throughout, via `lib/supabase/*`
Supabase Management API	Auth settings, the redirect allow-list, the e-mail templates and the SMTP server are **applied as code rather than clicked in a dashboard**, and are therefore reviewable and repeatable	Personal access token, held locally and never deployed	`scripts/auth-config.mjs`
Google Gemini (`@google/genai`)	The listing-photo check: whether an image genuinely shows the apartment, and which room it shows	`GEMINI_API_KEY`, server-side only	`lib/photo-vision.ts`
Gmail SMTP (`nodemailer`)	Sign-up codes, password reset, e-mail change, and the opt-in new-match alert	Gmail app password, server-side only	`lib/mail.ts`, `lib/auth-mail.ts`
Nominatim (OpenStreetMap)	Turning a typed street address into a map pin	None	`lib/geocode.ts`
Overpass (OpenStreetMap)	The cafés, restaurants, bars and shops shown near a room	None	`app/api/places/route.ts`
CARTO basemaps	Vector map tiles, in a light and a dark style	None	`components/ui/map.tsx`
Google Calendar + OAuth 2.0	Writing a confirmed viewing into the member's own calendar	Client id and secret	`lib/google.ts` — **built, deliberately not configured**

Two of these rows deserve a sentence of their own.

The application sends its own authentication mail. Supabase Auth is asked to *mint* the code — `admin.generateLink()` returns the same one-time code GoTrue would have mailed, and sends nothing — after which `lib/mail.ts` sends the message itself. The reason is deliverability: GoTrue's mailer offers no plain-text part, and an HTML-only message is one of the strongest junk signals a small sender can give. Measured on production from one account minutes apart, the multipart message reached the inbox while the HTML-only one did not. The Supabase templates remain uploaded and in step, as the fallback should these calls ever fail closed.

Google Calendar is present in the code and dormant in the deployment. The OAuth flow, the token table and the event write are all implemented, but no client credentials are set on the host, so `google_tokens` holds no rows and the chat falls back to plain "Add to Google Calendar" links. This is a deployment decision, not an unfinished feature.

Every outbound call is written to survive the service being unavailable

An external API which is down must degrade the feature that uses it, and must never take a page or an action down with it.

- **Gemini** is asked through a list of four models in order — three Flash models by judgement, then Flash-Lite for availability. The lite model is measurably blunter, and is still a better answer than "try again in a moment".

- **Overpass** is asked across three mirrors, hedged: a mirror gets 3.5 seconds to itself before the next is also asked, under a 12-second overall deadline. The main instance rate-limits, and cut the project off entirely while this was being built. A success is cached for a day; a failure is cached for nothing at all, so the next request genuinely asks again.
- **Nominatim** is keyless, and its usage policy asks for a real `User-Agent` and at most one request per second — which a listing form comfortably respects.
- **SMTP** never throws. A failed e-mail returns `false` and is logged; publishing a room succeeds whether or not the alert goes out.
- **With no SMTP configuration at all** the mailer becomes a no-op, which is what allows local development, CI and the test suite to run without ever reaching a mail server.

---

6. Core business logic

6.1 Compatibility scoring (`lib/compatibility.ts`)

There are two independent scores, and both of them range from 0 to 100.

Lifestyle is a weighted sum whose weights add up to 100:

COMPONENT	WEIGHT	COMPONENT	WEIGHT
Budget	20	Sleep schedule	6
City	18	Guests	6
Move-in date	10	Noise	4
Smoking	10	Diet	4
Cleanliness	10	Shabbat	4
Pets	8		

Social is the overlap between the declared interests of the two sides.

Two conventions carry most of the design:

1. **A missing preference receives approximately 60% of its weight rather than zero.** The statement "I did not answer" is not equivalent to the statement "we disagree". Scoring it as a mismatch would penalize incomplete profiles and would cause the deck to reward the filling of forms rather than genuine fit.
2. **Scoring is directional.** The `Perspective` parameter is either `"seeker"` or `"lister"`, because each side evaluates *its own* preferences against *the reality of the other side*. A seeker who tolerates guests, together with a household which hosts guests frequently, constitutes a good match from the point of view of the seeker; the reverse question is a different question entirely.

6.2 The deck (`lib/swipe.ts` , `lib/swipe-deck.ts`)

1. **Hard filters.** The city of the room must be one of the preferred cities of the seeker. In addition, the system excludes rooms which were already swiped, rooms belonging to the seeker, rooms which are paused, removed or taken, and anything belonging to a blocked member.
2. **Scoring** of both dimensions.
3. **Threshold:** if `sortKey(lifestyle, social) < MIN_DECK_SCORE (60)`, the room is dropped entirely.

4. **Ranking** by `sortKey`, with a cap at `DECK_SIZE`, which equals 60.

The threshold is the product decision. It is the reason that the deck may legitimately be empty, and therefore the reason that a "no strong matches yet" state had to be designed properly rather than treated as an error condition.

The division into two modules is a real technical constraint and not merely tidiness. The module `lib/swipe.ts` is imported by client components and must therefore be safe inside a browser bundle, whereas `lib/swipe-deck.ts` is marked `server-only` and holds the Supabase client which reads cookies.

6.3 Household semantics

- One person has one home: a unique index guarantees that an owner has at most one active listing.
- The owner of a listing cannot be reassigned through a normal `UPDATE`, because a trigger refuses this. Only the account-deletion handover, which opens a transaction-local GUC, is permitted to perform it.
- Deleting an account transfers a co-owned listing to an eligible roommate instead of cascading it away.

---

7. State management

The project contains **no state management library**. There is no Redux, no Zustand and no React Query. This is a deliberate choice, and the reasoning behind it is the clearest example of the model of the App Router producing a real benefit:

KIND OF STATE	WHERE IT LIVES
Server data	Server Components. It is not client state at all; it is rendered output.
Cached server data	Next.js Cache Components, using `"use cache"` for shared data and `"use cache: private"` for per-browser data, invalidated by tag.
Form state	`useActionState`, wrapped by `useStickyForm`, so that a rejected form retains whatever the user typed.
Ephemeral UI state	`useState`, used for open dialogs, the current swipe card and filter drafts.
URL state	`searchParams`, used for filters, sorting, page number and the active tab. It is therefore shareable and correct with respect to the back button by construction.
Realtime	A Supabase subscription which invalidates a cache tag, after which the server re-renders.

Adding a client cache library would have meant maintaining a second copy of the data of the server, together with its own separate invalidation rules. The cache-tag system already performs that role, in a single place (`lib/cache-tags.ts`), while the server remains the single source of truth.

The caching layer

There are five cached reads, each of which has a tag and a lifetime:

READ	CACHE	TAG
`queryListings`	shared	`listings`
`getSavedListingIds`	private	`saved:<userId>`
`getProfileTabData`	private	`profile:<userId>`
`getCachedDeck`	private	`deck:<userId>`
`getCachedInbox`	private	`chat:<userId>`

Results produced under `"use cache: private"` live **only in the memory of the requesting browser** and are never written into a shared server store, which is exactly what makes it safe to cache the deck and the inbox of an individual member. Every mutation calls `updateTag` for precisely what it modified, and therefore adding a room to the liked list does not discard the chats of that member.

---

8. Error handling

Server Actions return errors; they do not throw them. Every action returns a discriminated union, either `{ ok: true, … } | { ok: false, error: string }` or `{ error?: string }`, so that the form is able to render the message next to the relevant field. A thrown error would produce an error boundary, which is the correct response to a bug but the incorrect response to a wrong password.

Error handling is layered:

1. **Database constraints and RLS** form the last line of defense. A refused write returns an error code which the action translates. For example, `delete_own_account` raises the hints `pick_heir` and `bad_heir`, which the action converts into the message "Choose which roommate takes over your listing."
2. **Server Actions** validate using Zod, return field-level messages, and never expose internal details. As a special case, `sendMessageAction` treats Postgres error `23505`, which indicates a unique violation, as *success*, because that identifier had already been delivered.
3. **Route Handlers** return JSON together with an appropriate status code, and specifically 401 for unauthenticated API callers.
4. **React error boundaries**, for example `app/(public)/browse/error.tsx`.
5. **Deliberate non-failures.** The function `getUnreadCount()` catches everything and returns 0. It is only a decoration on a badge and is handed to the layout without being awaited; an unhandled rejection there would bring down an entire page merely because of a number.

Error messages state what the user should do next. The message "That code is wrong or has expired. Check the email, or send a new one" is considerably more useful than the message "Invalid OTP".

---

9. Input validation

Every input is validated twice, in two different places, and this is intentional.

1. **Client side.** HTML constraints together with immediate feedback. This exists for convenience only and is assumed to be bypassable.

2. **Server side, using Zod.** There are six schemas in `lib/validation/`, namely `profile`, `listing`, `message`, `filters`, `about` and `report`. The Server Action parses the input before touching the database. The TypeScript type is *inferred from the schema*, and therefore validation and types cannot drift apart.
3. **Database level.** `CHECK` constraints are used (`age between 18 and 120`, `bio <= 500`, `cleanliness between 1 and 5`, `budget_max = 0 or budget_max >= budget_min`), together with enums, foreign keys and unique indexes.

The following hardening measures are worth naming explicitly:

- **URL parameters are parsed rather than trusted.** The schema `listingFiltersSchema` uses `.catch()` throughout, so that a manually edited parameter such as `?safe_room=nonsense` falls back to the value "unset" instead of producing an error.
- **Open redirects are prevented.** The function `sanitizeNextPath()` guards every `?next=` parameter, and only same-site paths survive it.
- **File uploads.** The type and the size are checked on the client and then re-checked on the server. Furthermore, a chat photo must reside under the storage prefix of its own conversation.
- **Photo content.** The action `checkAndUploadPhotoAction` sends the bytes to Gemini and stores the file **only if** the image genuinely matches its declared tag. Consequently a bedroom slot cannot contain a picture of a car.

---

10. UX design

Principles

- **Mobile first.** Most people search for a room using a phone. The interface therefore uses a floating pill bottom navigation, and the navigation hides itself while a chat thread is open so that the composer owns the bottom edge of the screen.
- **Two themes.** Editorial, which is the light theme, and Noir, which is the dark theme and in which the accent color becomes gold. Both are driven entirely by semantic tokens.
- **A single typeface**, namely Inter, is used everywhere.
- **Nothing renders as "empty by accident".** Every empty state explains what happened and what the user should do about it. An empty deck is the result of a strict filter and not the result of a broken page.

Navigation

There are four destinations: **Swipe, Listings, Chat and Profile.** Movement from one tab to another slides left or right according to the tab order, using the View Transitions API, whereas every other navigation performs a crossfade.

Perceived performance

The pages are structured so that the static frame is sent first and only the member-specific portion is streamed afterwards:

- The layouts read no session at all, and therefore the shell is included in the prerender.
- Data-heavy sections are placed behind `<Suspense>` together with skeletons shaped like the real content, so that the replacement is experienced as a fill rather than as a jump.
- Cached reads travel together with the prefetched App Shell, and consequently returning to Swipe, Chat or Profile takes **approximately 60 ms with no skeleton at all**, as measured on production.

Deliberate restraint

No map is drawn until an icon is pressed. MapLibre is a large dependency, and the majority of visits never open a map. Loading it eagerly would therefore impose a cost on every visit for the sake of a feature which is used only on some of them.
